

Renforcement Java

Mickaël Foursov¹

¹ISTIC
Université de Rennes 1

L2 Miage Cocody, avril/mai 2024

Définition

Un **algorithme** est la description des opérations qui permettent d'accomplir une tâche. Cette tâche peut être un calcul, une transformation d'information, voire un procédé de fabrication.

Définition

Un **langage de programmation** est un langage rigoureux, compréhensible par une machine, qui permet d'exprimer **sans aucune ambiguïté** des algorithmes.

Définition

Un **programme** est un algorithme exprimé dans un langage de programmation.

La conception de programmes exige une **rigueur absolue**.

Pour assurer cette rigueur, les langages de programmation ont une **syntaxe** et une **sémantique** précises :

- La **syntaxe** est l'ensemble des règles de construction que doivent respecter les textes des programmes.
- La **sémantique** est la signification des constructions du langage.

```
while (i <= n) {  
    somme = somme + i ; i = i + 1;  
}  
while (i <= n) {  
    somme = somme + i i = i + 1;  
}  
while (i < n) {  
    somme = somme + i ; i = i + 1;  
}
```

Résolution d'un problème

- L'**analyse** : elle consiste à exprimer la résolution du problème sous une forme la moins ambiguë possible compréhensible par un être humain.
- La **programmation** : elle exprime cette solution en termes d'objets informatiques et d'instructions dans un langage de programmation.

Qualités d'un programme

- correction
- efficacité
- maintenabilité
- clarté du code

Exemple introductif : cahier des charges

Cahier des charges

Calculer la surface et le volume de plusieurs sphères de rayons donnés.

Analyse :

aire : réel $\rightarrow$ réel

$$\text{aire}(\text{rayon}) = 4\pi\text{rayon}^2$$

volume : réel $\rightarrow$ réel

$$\text{volume}(\text{rayon}) = 4\pi\text{rayon}^3/3$$

Exemple introductif : programme Java

```
public class Sphere {  
    static final double PI = 3.141592;  
    static double aire(double r) {  
        return 4 * PI * r * r;  
    }  
    static double volume(double r) {  
        return 4 * PI * r * r * r / 3;  
    }  
    public static void main(String[] arg) {  
        System.out.println(aire(4.2));  
        System.out.println(volume(4.2));  
        System.out.println(aire(2));  
        System.out.println(volume(2));  
    }  
}
```

Un **type** est caractérisé par :

- un domaine de valeurs
- les opérations qui sont possibles sur ces valeurs

- Cohérence des données

`12/"bonjour"`

`(i<14) - 12.1`

- Nécessité de représenter correctement l'information
- Intérêt de déterminer les besoins en mémoire

Panorama des diverses catégories de types

- Types primitifs
 - Type scalaires
 - entiers, p.e. 12
 - réels, p.e. 12.0
 - caractères, p.e. 'w'
 - booléens, p.e. false
 - Types composés
 - tableaux, p.e. [1, 2, 10]
 - chaînes de caractères, p.e. "coucou"
- Types programmés
 - énumérés
 - structures
 - classes

Définition

Une **déclaration** consiste à donner un nom et indiquer le type de donnée ou d'une fonction.

- type nom ;
 nom = expression ;
- type nom = expression ;
- typeDuResultat nomDeLaFonction(type nom, ...)

Description des types scalaires

- entiers : **int** (sur 32 bits, dans $[-2^{31}; 2^{31} - 1]$), **long** (64 bits), **short** (16 bits), **byte** (8 bits).

Opérations : +, -, *, /, %, ==, !=, >, <, >=, <=.

- nombres réels : **float** (sur 32 bits), **double** (sur 64 bits).
Le double écrit 6.02E23 vaut $6,02 \times 10^{23}$.

Opérations : +, -, *, /, >, <.

- caractères : **char** (p.e. 'x', '\n', \t).

Opérations : ==, !=, >, <, >=, <=.

- booléens : **boolean** (true et false).

Opérations : !, ||, &&, ==, !=.

Priorités des opérateurs (par ordre décroissant) :

moins unaire, {*, /, %}, {+, -}, {==, !=, >, <, >=, <=}, &&, ||.

```
enum Couleur {bleu, blanc, rouge};
```

On désigne les valeurs au moyen de la notation
`nomDuType.identificateur`.

```
Couleur milieuDuDrapeau = Couleur.blanc;  
milieuDuDrapeau == Couleur.blanc vaut true
```

Structure de programmes simples

```
class NomDuProgramme {  
    déclarations de données globales  
    définitions de fonctions  
    définition de la fonction principale (main)  
    // commentaires sur une ligne  
    /* commentaires  
    sur plusieurs  
    lignes */  
}
```

Définition

Une donnée est quelque chose qui :

- joue un certain **rôle**
- a un **nom** (ou **identificateur**)
- possède une valeur d'un certain **type**

Une donnée peut être **globale** ou **locale**.

Une donnée peut être **variable** ou **constante**.

Définitions de fonctions

Format général :

```
static typeRésultat nomFonction(type1 param1, type2 param2,...){  
    \\ déclarations locales et instructions  
    return expression;  
}
```

Exemples particuliers :

```
static typeRésultat nomFonction() {...}  
static void nomFonction(type1 param1,...) {...}
```

Cas particulier pour la fonction main

```
public static void main (String[] args) {...}
```

- On les place juste après l'en-tête de la fonction.
- Ils sont composés de 3 rubriques, présentes seulement si elles sont nécessaires :
 - **pré-requis** : conditions dans lesquelles l'appel est permis.
 - **effet** : effets sur l'extérieur ou sur certaines variables globales.
 - **résultat** : valeur rendue en résultat, en fonction des paramètres et, éventuellement, de l'extérieur et de certaines variables globales.

Les **fonctions** d'affichage de valeurs en console sont :

- `void System.out.print( diversType x)`
- `void System.out.println( diversType x)`

L'**opération de concaténation** de chaînes de caractères est +
Exemples :

- `"tant va l" + "a cruche à l'eau"`
- `int k = 12;`
`String s = "j'ai " + k + " oeufs dans le panier";`

Définition

Les noms qui apparaissent dans les programmes pour désigner les choses s'appellent des **identificateurs**. Ils doivent obéir à quelques règles simples :

- commencer par une lettre
- se poursuivre par des lettres ou des chiffres,
- ne pas être identiques à un mot-clef de langage.

Quelques conventions et conseils :

- choisir les noms les plus significatifs possible
`final int graviteTerrestre = 9.81`
- les classes commencent par une majuscule
`class TestMoyenne`
- Les données et fonctions commencent par une minuscule
`String prenom; int ageDuCapitaine;`
`int max(int i, int j){...}`

Portées d'identification

```
public class TestMachin {  
    static int v;  
    static double c;  
    static int f(int p) {  
        int cc; int x;  
        instructions utilisant x, cc, p, v, c  
    }  
    static void g(int p) {  
        char x;  
        instructions utilisant x, p, v, c  
    }  
    public static void main(String[] args) {  
        double y;  
        instructions utilisant y, v, c  
    }  
}
```

Exercice : qu'est-ce que ce programme affiche ?

```
public class TestPorteesIdentification {  
  
    static String texte = "alfred";  
  
    static void imprimeTexte() {  
        String texte = "jules";  
        System.out.println(texte);  
        texte = "dupont";  
        System.out.println(texte);  
    }  
    public static void main(String[] args) {  
        System.out.println(texte);  
        imprimeTexte();  
        System.out.println(texte);  
    }  
}
```

Instructions conditionnelles

Les instructions conditionnelles permettent de choisir les instructions à exécuter en fonction de certains critères. La forme la plus répandue permet de choisir d'exécuter une action parmi un ensemble d'actions en fonction de critères indiqués par des expressions booléennes.

```
if (condition1) {instructions1}  
else if (condition2) {instructions2}  
else if (condition3) {instructions3}  
...  
else {instructions}
```

Instructions conditionnelles

```
static int signe(int n) {  
    int resul;  
    if ( n > 0 ) { resul = -1; }  
    else if ( n == 0 ) { resul = 0; }  
    else /* n < 0 */ { resul = 1; }  
    return resul;  
}
```

```
static int max(int a, int b) {  
    if (a >= b) { return a; }  
    else /* a < b */ { return b; }  
}
```

Imbrication de conditionnelles

```
static int nombreDeJoursDuMois(int numeroDeMois) {  
    if (numeroDeMois < 8) {  
        if (numeroDeMois % 2 == 0) {  
            if (numeroDeMois==2) {return 28; }  
            else {return 30; }  
        }  
        else {return 31;}  
    }  
    else { // numeroDeMois >= 8  
        if (numeroDeMois %2 == 0) {return 31; }  
        else { return 30; }  
    }  
}
```

Switch (aiguillage)

```
switch (expression) {  
  case valeur1 : instructions1 break ;  
  case valeur2 : instructions2 break ;  
  case valeur3 : instructions3 break ;  
  ...  
  default : instructions ;  
}
```

Switch (aiguillage)

```
static int nombreDeJoursDuMois(int numeroDeMois){  
    switch (numeroDeMois) {  
        case 4 :  
        case 6 :  
        case 9 :  
        case 11 : return 30;  
        case 2 : return 28;  
        default : return 31;  
    }  
}
```

Récurtivité et répétition

La programmation atteint sa puissance en répétant un bloc d'instructions un nombre de fois non connu à l'avance.

La répétition est en effet le seul moyen d'exprimer de façon finie des quantités de calculs qui dépendent des données traitées.

```
static int somme(int n) {  
    if (n > 0) { return n + somme(n - 1); }  
    else /* n == 0 */ { return 0; }  
}
```

```
static int somme(int n) {  
    int s = 0;  
    int i = 0;  
    while (i < n) { s = s + i; i = i + 1; }  
    return s;  
}
```

$$\text{factorielle}(n) = n! = 1 \times 2 \times 3 \times \dots \times (n-1) \times n$$

```
static int factorielle(int n) {  
    // pré-requis : n > 0  
    // résultat : la factorielle de n  
    if (n == 1) { return 1; }  
    else { return (n * factorielle(n - 1)); }  
}
```

```
static int pgcd(int a, int b) {  
    // pré-requis : a > 0 et b > 0  
    // résultat : le pgcd de a et b  
    if (a == b) { return a; }  
    else if (a > b) { return pgcd(a - b, b); }  
    else /* a < b */ { return pgcd(a, b - a); }  
}
```

Itération : Boucle "tant que"

```
while (non condition d'arrêt) {  
    instructions (corps de la boucle)  
}
```

```
static int factorielle(int n) {  
    // pré-requis : n > 0 et n pas trop grand  
    // résultat : factorielle de n  
    int k = 1;  
    int fac = 1;  
    while ( k <= n ) {  
        fac = fac * k;  
        k = k + 1;  
    }  
    return fac;  
}
```

Itération : Boucle "tant que"

```
static int pgcd(int a, int b){  
    // pré-requis : a > 0 et b > 0  
    // résultat : le pgcd de a et b  
    int u = a; int v = b;  
    while ( u != v ) {  
        if ( u > v ) { u = u - v; }  
        else /* u <= v */ { v = v - u; }  
    }  
    return u;  
}
```

Itération : Boucle pour faire n fois

```
for (initialisation ; condition ; progression) {  
    instructions  
}
```

```
int somme = 0;  
for (int i = 0 ; i < 12 ; i = i + 1){  
    int nombreLu = Lecture.unEntier();  
    somme = somme + nombreLu;  
}
```

Remarque : attention à `++i` vs. `i = i + 1` vs. `i++`

Type structure

```
public class Point {  
    public double x;  
    public double y;  
}
```

```
public class CarteAJouer {  
    public ForceDeCarte force;  
    public CouleurDeCarte couleur;  
}
```

```
public class Personne {  
    public String nom;  
    public String age;  
    public Sexe sonSexe;  
}
```

```
enum CouleurDeCarte {trefle, carreau, coeur, pique}  
enum ForceDeCarte {sept,huit,neuf,dix,valet,dame,roi,as}
```

Type structure

```
public class Point {  
    public double x;  
    public double y;  
    public Point (double sonX, double sonY) {  
        x = sonX;  
        y = sonY;  
    }  
}
```

```
public class CarteAJouer {  
    public ForceDeCarte force;  
    public CouleurDeCarte couleur;  
    public CarteAJouer (ForceDeCarte f,  
                        CouleurDeCarte c) {  
        force = f;  
        couleur = c;  
    }  
}
```

Type structure

```
public class Personne {  
    public String nom, age;  
    public Sexe sonSexe;  
    public Personne (String n, int a, Sexe s) {  
        nom = n;  
        age = a;  
        sexe = s;  
    }  
}
```

Type structure

```
Point origine = new Point(0, 0);
Point p = new Point(1.0, 12.45);
CarteAJouer mistigri =
    new CarteAJouer(ForceDeCarte.valet,
                    CouleurDeCarte.trefle);
Personne toto =
    new Personne ("toto", 14, Sexe.masculin);
```

- Les objets **doivent** être créés (`new`) pour exister.
- Les objets sont **toujours désignés par référence**.
- **null** est la référence qui ne désigne aucun objet.

Définition

- Le type **chaîne de caractères** permet de représenter des suites de caractères de longueur quelconque.
- Chaque chaîne de caractères est un **objet** de type `String`.
- Elles sont donc **toujours désignées par référence**.

Attention :

- Comparaison de chaînes :
`chain1 == chain2` compare les références !!!
`chain1.equals(chain2)` compare le contenu des deux chaînes, caractère par caractère.
- Les chaînes de caractères sont des objets **non modifiables**.

Opérations sur les chaînes de caractères

- Chaîne vide (de longueur 0) : ""
- Concaténation : `s1 + s2`
- Longueur d'une chaîne `s` : `s.length()`
- N-ème caractère d'une chaîne `s` : `s.charAt(n)`
- Ordre alphabétique : `s1.compareTo(s2)`
- Sous-chaîne de `s` entre les positions `i` et `j` :
`s.substring(i, j)`
- Première position d'un caractère `c` dans une chaîne `s` :
`s.indexOf(c)`
- Première position de `c` dans `s`, à partir de la position `i` :
`s.indexOf(c, i)`

Programmer avec les chaînes

```
public class TexteEnMajuscule {
    static final String minuscules =
        "abcdefghijklmnopqrstuvwxyz";
    static final String majuscules =
        "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    static char majuscule(char c) { // ...
        int k = minuscules.indexOf(c);
        if (k == -1) { return c; }
        else { return majuscules.charAt(k); }
    }
    static String texteMajuscule(String txt) { //...
        String resul = "";
        for (int i = 0 ; i < txt.length() ; ++i) {
            resul += majuscule(txt.charAt(i));
        }
        return resul;
    }
    public static void main(String[] args) {
        System.out.println(texteMajuscules("test"));
    }
}
```

Définition

Un tableau regroupe une **collection de données de même type**, chaque élément étant **désigné à l'aide d'un indice**.

L'usage d'un tableau est justifié dans les cas suivants :

- Lorsque la quantité de données est importante : il n'est pas question de rédiger un programme qui définisse autant de noms individuels pour manipuler ces données.
- Lorsqu'on doit déterminer **par calcul** quels éléments de la collection sont utilisés.

Déclaration :

```
TypeDesElements[] nomDeTableau;
```

Création d'un tableau :

```
new TypeDesElements[taille];
```

Exemple :

```
int[] nomDuMois = new String[12];
```

Attention à l'initialisation du contenu du tableau !

En Java, les tableaux sont toujours indicés à partir de 0.
L'élément i d'un tableau `tab` se note `tab[i]`.

Les tableaux sont modifiables en Java :

```
nomDuMois[0] = "janvier";  
nomDuMois[1] = "février";  
...  
nomDuMois[11] = "décembre";
```

Attention : un tableau est désigné par une référence, et une fonction peut modifier un tableau passé en paramètre.

Déclarations de tableaux initialisés

```
int[] dixPremiersEntiers = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
```

```
static String nomDuMois(int n) {  
    // pré-requis : n >=1 et n <= 12  
    // résultat : le nom du mois numéro n  
    final String[] mois = {  
        "janvier", "février", "mars", "avril", "mai",  
        "juin", "juillet", "août", "septembre",  
        "octobre", "novembre", "décembre"};  
    return mois[n - 1];  
}
```

Déclarations de tableaux initialisés

```
public class TestMax {  
    public static int max(int[] T){  
        // pré-requis : T.length > 0  
        // résultat : le maximum des éléments de T  
        int resul = T[0];  
        for (int i = 1 ; i < T.length ; ++i) {  
            if (T[i] > resul) {  
                resul = T[i];  
            }  
        }  
        return resul;  
    }  
    public static void main(String[] arg){  
        int[] T1 = {12, 56, 4, 89, -23, 0, 56};  
        System.out.println("le max de T1=" + max(T1));  
        int[] T2 = {34, -23, 23, 45, 4, 3};  
        System.out.println("le max de T2=" + max(T2));  
    }  
}
```

Programmation sur les tableaux

```
static int recherche(int[] T, int val) {  
    //résultat: la pos de val dans T, si absent -1  
    int i = 0;  
    while ( i < T.length &&  T[i] != val) {  
        ++i;  
    }  
    if (i == T.length) {  
        return -1;  
    } else {  
        return i;  
    }  
}
```

Tableaux à plusieurs dimensions

Un tableau à plusieurs dimensions est un tableau dont les éléments sont désignés par plusieurs indices.

```
PieceDEchecs[][] echiquier = new  
                                PieceDEchecs[8][8];  
echiquier[4][6] = new PieceDEchecs(...)
```

Programmer avec des tableaux multi-dimensionnels

```
public class TestMatrice {  
    static int[][] produitMatrice(int[][]A,int[][]B){  
        //pré-requis: A est matrice n x p, B une p x q  
        //résultat : le produit matriciel A x B  
        final int n = A.length;  
        final int p = B.length;  
        final int q = B[0].length;  
        int[][] C = new int[n][q]; // matrice résultat  
        for (int i = 0; i < n ; ++i) {  
            for (int j = 0; i < q ; ++j) {  
                C[i][j] = 0;  
                for (int k = 0; i < p ; ++k) {  
                    C[i][j] = C[i][j] + A[i][k] * B[k][j];  
                }  
            }  
        }  
        return C;  
    }  
    public static void main (String[] args) { ... }  
}
```

Objets de type classe

```
public class Robot {
    public int X; public int Y; public Orientation o;
    public Robot (int x, int y, Orientation versOu) {
        X = x ; Y = y ; o = versOu ;
    }
    public Robot () { X = 0; Y = 0; o = Orientation.nord; }
    public void avancer () {
        switch(o) {
            case nord : ++Y ; break;
            case est  : ++X ; break;
            case sud  : --Y ; break;
            case ouest: --X ; break;
        }
    }
    public void tournerADroite() {
        switch(o) {
            case nord : o = Orientation.est ; break;
            case est  : o = Orientation.sud ; break;
            case sud  : o = Orientation.ouest ; break;
            case ouest: o = Orientation.nord ; break;
        }
    }
}
```

Objets de type classe

- `Robot vigor=new Robot(1,3,Orientation.nord);`
`vigor.avancer();`

- Instance courante (`this`)

- `static` vs. non statique

Les composants statiques d'une classe **existent en un seul exemplaire** et ne sont pas associés à un objet particulier.

Les composants non statiques **existent dans chaque objet, et lui sont propres.**

Les propriétés du robot doivent être les mêmes quelle que soit la façon dont il est concrètement représenté. Exemples :

- Tourner ne doit pas modifier les coordonnées
- Avancer ne doit pas modifier l'orientation

Encapsulation

```
public class Robot {  
    private int X;  
    private int Y;  
    private Orientation o;  
  
    public Robot(int x,int y,Orientation u){...}  
    public Robot () {...}  
  
    public void avancer () {...}  
    public void tournerADroite () {...}  
  
    public int getX () { return X; }  
    public int getY () { return Y; }  
    public Orientation getOrientation(){  
        return o;  
    }  
}
```

Programmer avec des objets

```
public class Personne {  
    public static final int tailleNormaleAdulte = 170;  
    public static int plusGrandeTaille = 0;  
    private String nom;  
    private int taille;  
    public Personne (String ceNom, int cetteTaille) {  
        nom = ceNom; taille = cetteTaille;  
        if (cetteTaille > plusGrandeTaille) {  
            plusGrandeTaille = cetteTaille;  
        }  
    }  
    public String getNom () { return nom; }  
    public int getTaille () { return taille ; }  
    public setTaille (int nouvelleTaille) {  
        taille = nouvelleTaille;  
        if (nouvelleTaille > plusGrandeTaille) {  
            plusGrandeTaille = nouvelleTaille;  
        }  
    }  
    public static int getPlusGrandeTaille () {  
        return plusGrandeTaille;  
    }  
}
```

Programmer avec des objets

```
public class TestPersonne {
    public static void main(String[] arg) {
        Personne jules = new Personne("Jules", 135);
        Personne urbain = new Personne("Urbain", 183);
        Personne andronique = new Personne("Andronique", 168);

        int k = Personne.getPlusGrandeTaille();
        System.out.println("La plus grande taille est "+k);

        jules.setTaille(183);
        int k = Personne.getPlusGrandeTaille();
        System.out.println("La plus grande taille est "+k);
        if (k > Personne.tailleNormaleAdulte) {
            System.out.println("C'est plus que la normale");
        }
    }
}
```